

MOVEDATA NYC: Urban Mobility Analysis Using NYC Taxi Data

1. Problem Framing and Dataset Analysis

Every day, millions of people travel around New York City using different forms of transportation. Among these, taxis remain one of the most common ways people move between neighborhoods, airports, business districts, and residential areas. Because of this, taxi trip data provides a useful way to understand how people travel, how traffic behaves, and how different parts of the city are connected.

The goal of this project was to build a full-stack application that could process, store, analyze, and visualize New York City taxi trip data. Instead of simply displaying raw numbers, we wanted to create a system that helps users explore urban mobility patterns and discover meaningful insights about how the city moves.

To achieve this, we worked with three official NYC Taxi and Limousine Commission (TLC) datasets:

- Yellow Taxi Trip Records
- Taxi Zone Lookup Table
- Taxi Zone Spatial Metadata

The trip records dataset contains millions of taxi trips and includes information such as pickup and dropoff times, trip distance, passenger count, fare amount, tip amount, and location IDs. The lookup table helps translate location IDs into actual zone names and boroughs, while the spatial dataset contains the geographic boundaries needed for mapping and visualization.

Data Challenges

One of the first things we discovered was that working with real-world data is much more difficult than working with small classroom datasets. Although the NYC Taxi dataset is very rich in information, it also contains inconsistencies that must be handled before analysis can begin.

Some of the issues we encountered included:

- Missing values in several fields

- Duplicate records
- Trips with invalid timestamps
- Negative or unrealistic distance values
- Outliers that could affect calculations and visualizations

To solve these problems, we created a cleaning process that removed invalid records while keeping track of everything through a transparency log. This ensured that our analysis was based on reliable and trustworthy data.

Feature Engineering

After cleaning the dataset, we created additional features that were not directly provided in the original data.

The first feature was **Trip Duration**, which was calculated by finding the difference between pickup and dropoff times.

The second feature was **Average Speed**, calculated using trip distance and trip duration. This allowed us to study traffic conditions and compare travel efficiency across different locations.

The third feature was the **Tip-to-Fare Ratio**, which measures how much a passenger tipped relative to the fare amount.

Finally, we created a **Rush Hour Indicator** to identify trips that occurred during peak traffic periods. This helped us compare normal traffic conditions with rush-hour traffic.

These new features played an important role in the later stages of our analysis.

Unexpected Observation

One of the most surprising findings from our project was that traffic congestion had almost no effect on passenger tipping behavior. Before conducting the analysis, we expected passengers stuck in heavy traffic to tip differently from passengers on faster trips. However, the results showed almost no relationship between trip speed and tipping percentage.

This finding eventually became one of the main insights explored in our project.

2. System Architecture and Design Decisions

To manage such a large dataset efficiently, we designed our project using a layered architecture. Separating the system into different components made development easier and allowed each team member to focus on a specific area of the project.

System Flow

NYC Taxi Dataset



ETL Processing Pipeline



PostgreSQL/PostGIS Database



Backend API



Frontend Dashboard

The ETL pipeline was responsible for extracting, cleaning, and transforming the data before loading it into the database. The backend then provided access to the processed data, while the frontend presented the results through an interactive dashboard.

Technologies Used

Throughout the project, we used several technologies that worked together to create the final application.

Backend

- Python
- Pandas
- GeoPandas
- SQLAlchemy

Database

- PostgreSQL
- PostGIS

Frontend

- HTML

- CSS
- JavaScript
- Leaflet

Each technology was selected because it addressed a specific need within the project, whether that was data processing, database management, spatial analysis, or visualization.

3. Database Design

The database was designed by separating the data into two main tables.

The first table, **nyc_taxi_zones**, stores information about taxi zones throughout New York City. This includes the borough name, zone name, service zone category, and geographic boundary.

The second table, **high_volume_trips**, stores the taxi trip records themselves.

Instead of storing borough names and zone names repeatedly for millions of trips, the trip table references the zone table through foreign keys. This approach reduces duplicate information and follows normalization principles.

Data Integrity

Several constraints were added to protect the quality of the stored data.

For example:

- Trip distances cannot be negative.
- Fare totals cannot be negative.
- Passenger counts must remain within realistic limits.
- Dropoff times must occur after pickup times.
- Rush-hour indicators can only contain valid values.

These checks help prevent invalid records from entering the database.

Database Performance

Because the dataset contains millions of records, performance was an important consideration.

Indexes were created on frequently queried fields such as:

- Pickup time
- Pickup zone
- Dropoff zone
- Combined pickup zone and time

These indexes allow the system to retrieve information much faster when users apply filters or interact with the dashboard.

Spatial Data Support

Since the assignment included taxi zone boundary data, PostGIS was enabled within PostgreSQL.

PostGIS allowed us to store geographic boundaries directly inside the database and made it possible to display taxi zones on an interactive map. A spatial index was also created to improve the speed of map-based queries.

4. Algorithmic Logic and Data Structures

One of the goals of our analysis was to determine whether traffic conditions influence passenger tipping behavior.

To investigate this question, we implemented a custom Pearson Correlation algorithm instead of relying on built-in statistical functions. This allowed us to demonstrate our understanding of algorithmic problem solving while satisfying the assignment requirement.

The algorithm compares two variables:

- Average Speed (MPH)
- Tip Percentage

The process begins by calculating the mean of both variables. It then computes the covariance and variance values required to calculate the Pearson Correlation coefficient.

```
Python

import math

def custom_pearson_correlation(x, y):
    if len(x) != len(y):
        raise ValueError("Lists must be of the same length.")
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    numerator = 0
    sum_sq_x = 0
    sum_sq_y = 0

    for i in range(n):
        diff_x = x[i] - mean_x
        diff_y = y[i] - mean_y
        numerator += diff_x * diff_y
        sum_sq_x += diff_x ** 2
        sum_sq_y += diff_y ** 2

    denominator = math.sqrt(sum_sq_x * sum_sq_y)
    if denominator == 0:
        return 0
    return numerator / denominator

print("Calculating Custom Pearson Correlation...")
speed_list = df_trips['average_speed_mph'].tolist()
tip_list = df_trips['tip_percentage'].tolist()

r_value = custom_pearson_correlation(speed_list, tip_list)
print(f"Custom Pearson Correlation (r): {r_value:.4f}")
```

Complexity Analysis

The algorithm runs in $O(N)$ time because it only requires two passes through the dataset and does not use nested loops.

Its memory usage is $O(1)$ because it only stores a small number of variables throughout execution.

Results

The algorithm produced a correlation coefficient of:

$r = -0.0080$

Since this value is extremely close to zero, the results indicate that there is almost no relationship between taxi speed and tipping behavior.

In simple terms, passengers appear to tip at roughly the same rate regardless of whether they experience heavy traffic or a relatively fast trip.

5. Insights and Interpretation

Insight 1: Most Taxi Trips Occur at Relatively Low Speeds

Analysis of trip speeds showed that most taxi rides occur at relatively low average speeds. This is not surprising given the amount of traffic found throughout New York City, especially in Manhattan.

```
plt.subplot(1, 3, 1)
sns.histplot(df_trips['average_speed_mph'], bins=50, kde=True, color='blue')
plt.title('Frequency Distribution: Average Speed')
plt.xlabel('Average Speed (mph)')
```

Insight 2: Tipping Behavior Is Surprisingly Consistent

When examining tip percentages, we noticed that many passengers tipped within a similar range regardless of travel conditions. This suggests that tipping habits may be influenced more by personal preference than by traffic conditions.

```
#graph 2 univariate frequency distribution (Tips)
plt.subplot(1,3,2)
```

```
#Limiting to 50% tip for better visibility of the main distribution
sns.histplot(df_trips[df_trips['tip_percentage'] <= 50]['tip_percentage'], bins=50,
plt.title('Frequency Distribution: Tip Percentage')
plt.xlabel('Tip Percentage (%)')
```

Activate W

Insight 3: Traffic Congestion Does Not Affect Tipping

The Pearson Correlation analysis revealed almost no relationship between traffic speed and tip percentage.

This was one of the most surprising findings of the project because it challenged our original assumption.

```
#graph 3 bivariate scatter plot (Testing Hypothesis A)
plt.subplot(1,3,3)
#plotting a random sample of 10,000 trips so the graph so it won't overload my pc
sample_df = df_trips.sample(1000, random_state=42)
sns.scatterplot(x='average_speed_mph', y='tip_percentage', data=sample_df, alpha=0.5)
plt.title('Bivariate: Speed vs. Tip Percentage')
plt.xlabel('Average Speed (mph)')
plt.ylabel('Tips Percentage (%)')
```

Insight 4: Travel Speeds Vary Across Boroughs

Statistical testing showed significant differences in travel speeds between boroughs.

This indicates that location plays a major role in determining traffic conditions and travel efficiency across New York City.

```
import scipy.stats as stats

print("Testing Hypothesis B: Speeds across different Boroughs")
```

```
# group the speeds by the Pickup Borough
boroughs = df_trips['PUBorough'].unique()
speed_groups = [df_trips[df_trips['PUBorough'] == borough]['average_speed_mph'] for borough in boroughs]

# Perform One-way ANOVA
f_stat, p_value = stats.f_oneway(*speed_groups)

print(f"ANOVA F-Statistics: {f_stat:.2f}")
print(f"ANOVA P-Value: {p_value}")

if p_value < 0.05:
    print("\nConclusion: p_value is practically 0. We REJECT the null hypothesis.")
    print("This proves that traffic congestion (average speed) differs significantly")
else:
    print("\nConclusion: We fail to reject the null hypothesis.")
```

Activate V

6. Frontend Dashboard

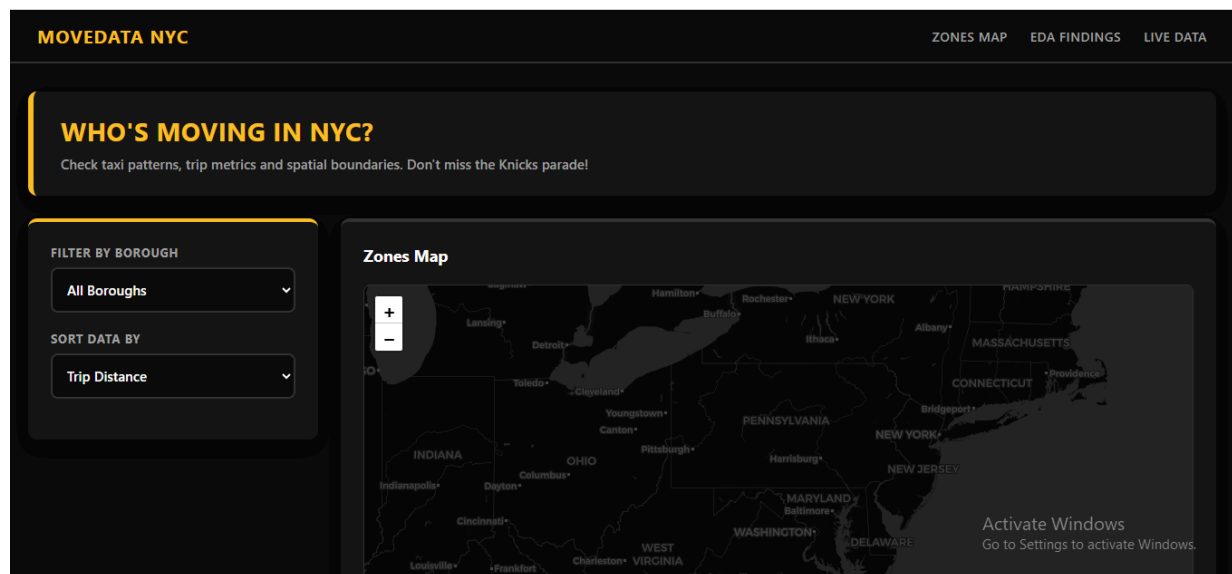
To make the analysis accessible and interactive, we developed the MoveData NYC dashboard.

The dashboard allows users to explore the dataset without needing to write queries or understand the underlying database structure.

Key features include:

- Borough filtering
- Trip metric sorting
- Interactive taxi zone map
- Exploratory Data Analysis findings
- Live data visualization

Users can interact with the map, apply filters, and view different visualizations to better understand mobility patterns across the city.



7. Reflection and Future Work

One of the biggest challenges we faced was working with a dataset containing millions of records. Processing such a large amount of data required careful planning, especially during the cleaning and loading stages.

Another challenge was integrating multiple datasets together while ensuring that all relationships remained accurate. Combining trip records, lookup tables, and spatial boundaries required extensive testing and validation.

Despite these challenges, the project gave us valuable experience in data engineering, database design, backend development, statistical analysis, and frontend visualization.

If we had more time, we would like to extend the system by adding:

- Real-time taxi data
- Weather integration
- Demand prediction models
- Advanced route analysis
- Additional borough comparison tools

Conclusion

This project allowed us to transform a large collection of raw taxi records into a complete data-driven application. Through data cleaning, feature engineering, database design, algorithm implementation, and interactive visualization, we were able to explore how people move throughout New York City and uncover patterns hidden within millions of taxi trips.

More importantly, the project showed how different areas of software development can work together to solve real-world problems and turn complex datasets into meaningful insights.